NETWORK PROTOCOL & APPLICATION — 01418351 NETWORK
SOCKET PROGRAMMING PROJECT

SMAP

SIMPLE MUSIC ADAPTIVE PROTOCOL

PROTOCOL OBJECTIVE

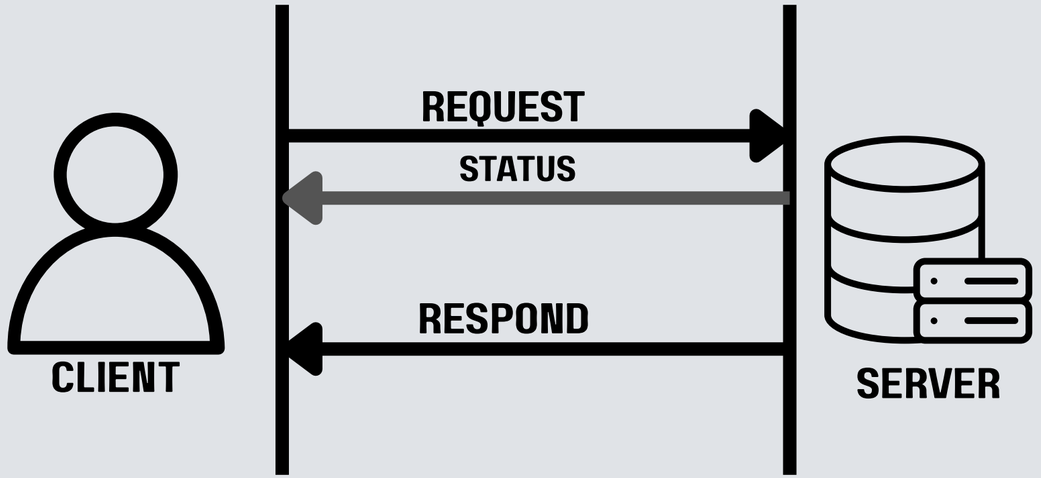
MAIN GOALS:

- STREAM MP3 AUDIO WITH ABR OVER A PERSISTENT TCP CONNECTION
- LIGHTWEIGHT LOWER OVERHEAD
- ADAPT BITRATE BASED ON MEASURED THROUGHPUT
- SUPPORT MULTIPLE SIMULTANEOUS CLIENTS
- PROVIDE CONTROLLABLE NETWORK CONDITIONS FOR TESTING

PROTOCOL CHARACTERISTICS

SMAP IS A CUSTOM, TEXT-BASED APPLICATION-LAYER PROTOCOL FOR MP3 STREAMING OVER TCP.

- APPLICATION-LAYER-PROTOCOL
- PERSISTENT CONNECTION
- STATEFUL PROTOCOL
- ADAPTIVE BITRATE



SMAP PROTOCOL ARCHITECTURE

END-TO-END COMMUNICATION FLOW	A CLEAN, LAYERED ARCHITECTURE RELYING ON A CUSTOM SMAP APPLICATION LAYER OPERATING OVER A PERSISTENT TCP TRANSPORT LAYER. THIS ENSURES RELIABLE, IN-ORDER DELIVERY OF AUDIO DATA, WHICH IS CRITICAL SINCE LOST OR REORDERED PACKETS WOULD CORRUPT THE MP3 STREAM.
WHY I USE TCP TRANSPORT LAYER	RELIABLE, ORDERED BYTE-STREAM DELIVERY IS DELEGATED TO TCP. SMAP LEVERAGES TCP CONNECTION PERSISTENCE TO MAINTAIN UNINTERRUPTED AUDIO STREAMING SESSIONS.

TCP BYTE STREAM & BUFFEREDSOCKET

TCP IS A BYTE STREAM, NOT A MESSAGE PROTOCOL; ONE MESSAGE CAN SPAN MULTIPLE RECV() CALLS.
BUFFEREDSOCKET: RESOLVES THIS BY BUFFERING INCOMING BYTES.
READ_UNTIL(): LOCATES DELIMITERS LIKE \N
READ_EXACT(): READS EXACT BYTE COUNTS BASED ON CONTENT-LENGTH

SMAP MESSAGE FORMAT

LIGHTWEIGHT, TEXT-BASED WIRE FORMAT

REQUESTS UTILIZE A SINGLE-LINE COMMAND STRUCTURE CONTAINING THE COMMAND NAME AND PARAMETERS (E.G., GET_SEGMENT 1 5 128).
RESPONSES FEATURE AN HTTP-LIKE STRUCTURE WITH A STATUS LINE (E.G., 200 OK), EXPLICIT HEADER FIELDS LIKE CONTENT-LENGTH, A BLANK LINE DELIMITER, AND THE RAW BINARY MP3 PAYLOAD.

SMAP COMMANDS

ONE PERSISTENT TCP CONNECTION PER CLIENT

COMMAND	PURPOSE
LIST	GET SONG CATALOG AND AVAILABLE BITRATES
PLAY	STREAM THE ENTIRE MP3 AT ONE BITRATE
GET_SEGMENT	REQUEST ONE MP3 SEGMENT
BITRATE	SET PREFERRED BITRATE
NETWORK	SET NETWORK SIMULATION PROFILE
STOP	RESET STREAMING STATE
QUIT	CLOSE THE TCP CONNECTION

STATUS CODE

STATUS CODE	STATUS PHRASE	MEANING (AS USED IN THE CODE)
200	OK	REQUEST SUCCEEDED
400	BAD REQUEST	MALFORMED REQUEST (WRONG PARAMETER COUNT, NON-INTEGER PARAMETER, EMPTY LINE, UNKNOWN COMMAND)
404	NOT FOUND	SONG ID DOES NOT EXIST, OR (FOR GET_SEGMENT) NO MORE SEGMENTS REMAIN FOR THAT BITRATE
406	NOT ACCEPTABLE	REQUESTED BITRATE IS NOT SUPPORTED, OR NOT AVAILABLE FOR THAT SPECIFIC SONG
500	INTERNAL SERVER ERROR	SONG FILE MISSING ON DISK, OR AN UNEXPECTED SERVER-SIDE EXCEPTION

SEGMENT-BASED STREAMING

FIXED-SIZE MEDIA SEGMENTATION

SOURCE MP3 FILES ARE PRE-SLICED INTO STANDARDIZED 8192-BYTE SEGMENTS RATHER THAN DYNAMICALLY GENERATED PER REQUEST. CLIENTS REQUEST ONE SEGMENT AT A TIME SEQUENTIALLY, CREATING THE FUNDAMENTAL MECHANISM THAT ALLOWS THE STREAM QUALITY TO ADAPT BETWEEN SEGMENT BOUNDARIES

ADAPTIVE BITRATE (ABR)

THROUGHPUT-DRIVEN QUALITY SELECTION

THE CLIENT ACTIVELY MEASURES THE REAL ELAPSED DOWNLOAD TIME FOR EACH SEGMENT TO CALCULATE CURRENT NETWORK THROUGHPUT IN KBPS. THE ALGORITHM AVERAGES THE LAST THREE MEASUREMENTS TO FILTER NETWORK NOISE, APPLIES A STRICT 70% SAFETY FACTOR, AND SELECTS THE HIGHEST SUSTAINABLE QUALITY FROM THE AVAILABLE 64, 128, 192, OR 320 KBPS BITRATES.

BITRATE SELECTION

DYNAMIC DECISION LOGIC

HOW DOES ABR CHOOSE A BITRATE?

THROUGHPUT FORMULA

THROUGHPUT=(BYTES×8)/(SECONDS×1000)

SELECTION RULES

- NEVER EXCEED PREFERRED BITRATE
- SELECT THE HIGHEST BITRATE SUSTAINABLE BY THE NETWORK
- MINIMUM BITRATE IS 64 KBPS

SUPPORTED BITRATE (64, 128, 192, 320)KBPS

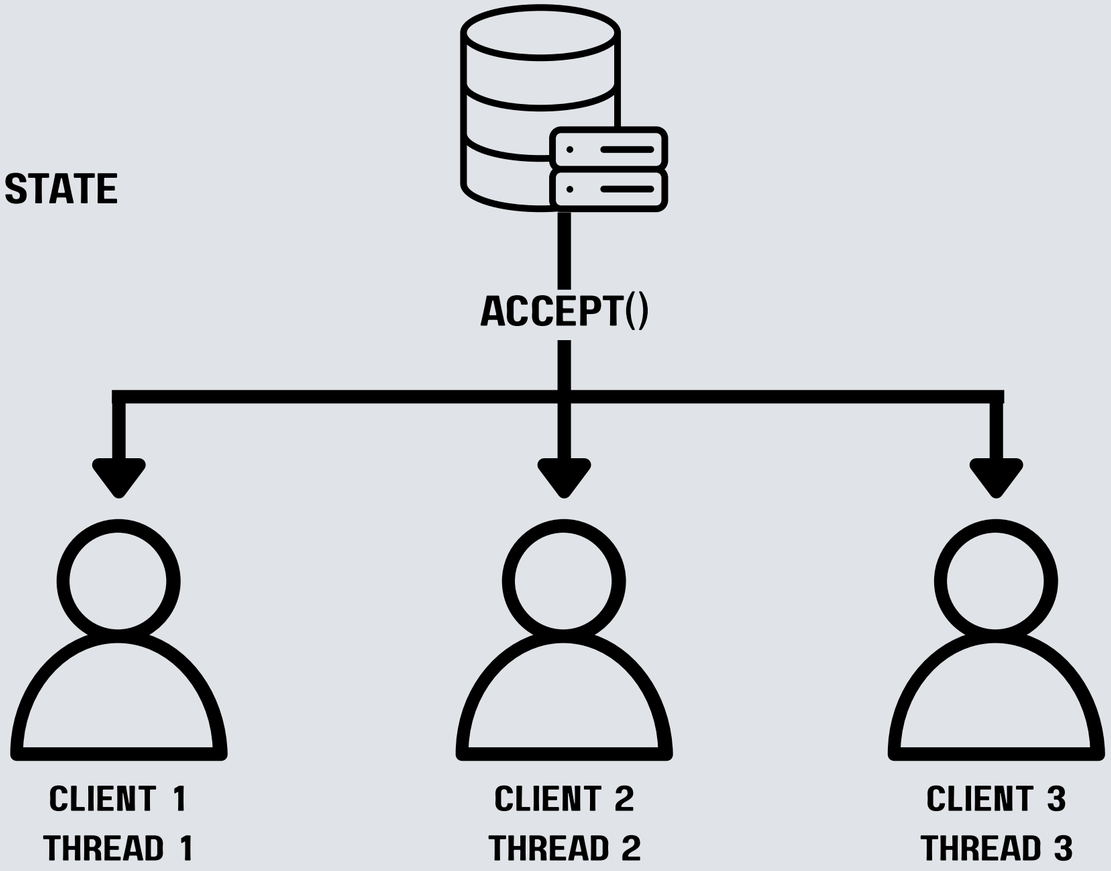
MULTI-CLIENT SUPPORT

DESIGN:

ONE CONNECTION → ONE THREAD → ONE SESSION STATE

EACH CLIENT HAS INDEPENDENT:

- **PREFERRED BITRATE**
- **STREAMING STATE**
- **NETWORK PROFILE**

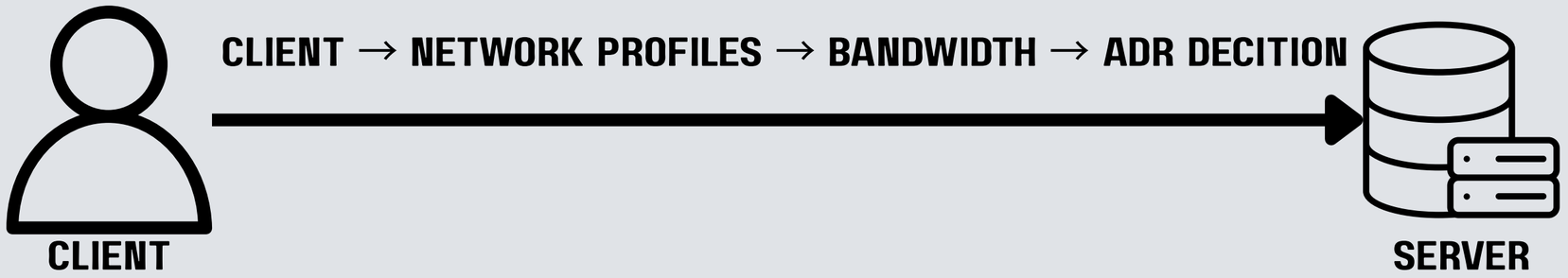


PER-CLIENT NETWORK CONDITIONS

NETWORK PROFILES:

- GOOD
- NORMAL
- POOR
- AUTO

INDEPENDENT NETWORK CONDITIONS



PROTOCOL'S CODE

CORE CLASSES

1 CLASS PROTOCOLERROR(EXCEPTION):

EXCEPTION CLASS SENDS AN ERROR MESSAGE THAT IS NOT FORMATTED

2 CLASS CONNECTIONCLOSEDERROR(EXCEPTION):

EXCEPTION CLASS TO TELL PEER THAT CONNECTION IS CLOSED

3 CLASS BUFFEREDSOCKET:

CORE WRAPPER CLASS TO FIX TCP BYTE STEAM BY CALL RECV()

PROTOCOL'S CODE

LOW-LEVEL SOCKET COMMUNICATION

PROTOCOL METHODS

1 SEND_ALL(SOCK, DATA: BYTES)

A WRAPPER FUNCTION UTILIZING SOCK.SENDALL() TO GUARANTEE THAT EVERY SINGLE BYTE OF DATA IS TRANSMITTED COMPLETELY

2 RECEIVE_EXACT(BUFFERED_SOCK, N: INT) -> BYTES

READS EXACTLY N BYTES FROM THE BUFFER, PROVIDING A CLEAN, FUNCTION-STYLE WRAPPER FOR BUFFERED_SOCK.READ_EXACT(N) TO SIMPLIFY DATA EXTRACTION BASED ON CONTENT-LENGTH

PROTOCOL'S CODE

REQUEST HANDLING (CLIENT-TO-SERVER)

PROTOCOL METHODS

1 PARSE_REQUEST(LINE: STR) -> (COMMAND, PARAMS)

SPLITS AN INCOMING REQUEST STRING INTO AN UPPERCASE COMMAND AND A LIST OF PARAMETERS

2 BUILD_REQUEST(COMMAND: STR, *PARAMS) -> BYTES

ASSEMBLES A COMMAND AND ITS PARAMETERS INTO A SINGLE STRING, ENCODES IT INTO BYTES, AND APPENDS A NEWLINE

3 SEND_REQUEST(SOCK, COMMAND: STR, *PARAMS) -> STR

COMBINES BUILD_REQUEST AND SEND_ALL TO INSTANTLY SEND A REQUEST OVER THE SOCKET, RETURNING THE SENT STRING FOR CLIENT-SIDE LOGGING

4 RECEIVE_REQUEST(BUFFERED_SOCK) -> (COMMAND, PARAMS, RAW_LINE)

READS A SINGLE LINE FROM THE TCP BUFFER, PASSES IT THROUGH PARSE_REQUEST, AND RETURNS THE PARSED DATA ALONGSIDE THE RAW TEXT FOR SERVER-SIDE LOGGING

PROTOCOL'S CODE

RESPONSE HANDLING (SERVER-TO-CLIENT)

PROTOCOL METHODS

1 **BUILD_RESPONSE(STATUS_CODE, HEADERS=NONE, BODY=NONE) -> BYTES**

CONSTRUCTS A PROPERLY FORMATTED SMAP RESPONSE BLOCK CONTAINING THE STATUS PHRASE, HEADERS, A BLANK LINE AND AN OPTIONAL BINARY BODY

2 **SEND_RESPONSE_HEADER(SOCK, STATUS_CODE, HEADERS=NONE) -> STR**

GENERATES ONLY THE HEADER BLOCK USING BUILD_RESPONSE AND TRANSMITS IT IMMEDIATELY VIA SEND_ALL FOR CASES WHERE THE BODY WILL BE STREAMED SEPARATELY

3 **RECEIVE_RESPONSE_HEADER(BUFFERED_SOCK) -> (STATUS_CODE, STATUS_PHRASE, HEADERS, RAW_TEXT)**

READS THE COMPLETE RESPONSE HEADER BLOCK UNTIL IT DETECTS \N, THEN SEPARATES THE STATUS CODE AND MAPS ALL HEADERS INTO A DICTIONARY FOR EASY CLIENT PARSING

PROTOCOL'S CODE

DATA CONVERSION & ADAPTIVE BITRATE (ABR)

PROTOCOL METHODS

1 **TO_JSON_BYTES(OBJ) -> BYTES**

CONVERTS PYTHON OBJECTS (SUCH AS THE SERVER'S SONG CATALOG) INTO UTF-8 ENCODED, READABLE JSON BYTES TO SERVE AS THE RESPONSE BODY FOR THE LIST COMMAND

2 **CALCULATE_THROUGHPUT_KBPS(NUM_BYTES, ELAPSED_SECONDS) -> FLOAT**

COMPUTES THE NETWORK DOWNLOAD SPEED IN KILOBITS PER SECOND USING THE THROUGHPUT FORMULA, IMPLEMENTING A 0.001-SECOND MINIMUM TIME SAFEGUARD TO PREVENT DIVISION-BY-ZERO ERRORS.

3 **SELECT_BITRATE(PREFERRED_BITRATE, AVAILABLE_THROUGHPUT_KBPS) -> (BITRATE, REASON)**

THE CORE ABR ALGORITHM THAT SELECTS THE HIGHEST AUDIO QUALITY THAT REMAINS STRICTLY BELOW THE USER'S PREFERRED_BITRATE CEILING AND WITHIN THE NETWORK'S AVAILABLE_THROUGHPUT_KBPS CONSTRAINT

DEMO